



Desafio Técnico

Desenvolvedor de Software

Página 1

Instruções para o desafio

- Crie um repositório Git para o versionamento do código.
- Assim que concluir o seu desafio, nos envie o link do repositório para a avaliação.

Contexto

Uma matriz curricular representa a oferta de aulas de um semestre, definida por um coordenador, e disponibilizada para matrícula dos alunos.

Neste desafio, não é necessário implementar o cadastro base de:

- Disciplinas (Mínimo 15);
- Professores (Mínimo 5);
- Horários (Mínimo 9);
- Cursos (Mínimo 9);
- Alunos (Mínimo 5);
- Coordenadores (Mínimo 3).

Essas entidades já devem existir no banco de dados, com dados fixos, e devem apenas ser consumidas pelo sistema.

O foco do desafio é a gestão da matriz curricular, controle de acesso e processo de matrícula, respeitando regras acadêmicas e de concorrência.

Objetivo

Desenvolver um sistema, backend e frontend, que permita:

- Coordenadores criarem e gerenciarem matrizes curriculares
- Alunos visualizarem e realizarem matrícula em aulas
- Garantir regras de negócio como:
 - Vagas limitadas
 - Restrições por curso
 - Conflito de horários
 - Controle de acesso por perfil



Desafio Técnico

Desenvolvedor de Software

Página 2

Perfis de Usuário

Coordenador

- Acessa apenas informações sob sua responsabilidade;
- Cria e gerencia matrizes curriculares.

Aluno

- Acessa apenas sua própria matrícula;
- Realiza matrícula nas aulas disponíveis para seu curso.

Funcionalidades

Acesso

- Deve haver login e logout utilizando o keycloak.

Coordenador

Criar matriz curricular (aula)

Uma aula da matriz curricular deve conter:

- Disciplina (pré-cadastrada);
- Professor (pré-cadastrado);
- Horário (pré-cadastrado);
- Lista de cursos autorizados;
- Quantidade máxima de alunos.

Regras:

- Uma mesma disciplina pode ser ofertada mais de uma vez, desde que em horários diferentes.
- A criação deve validar:
 - Existência das entidades referenciadas;
 - Horários válidos.

Visualizar matriz curricular

O coordenador pode listar e pesquisar as aulas da matriz usando os seguintes filtros:

- Intervalo de horário;
- Período do dia;



Desafio Técnico

Desenvolvedor de Software

Página 3

- Manhã;
- Tarde;
- Noite.
- Cursos autorizados;
- Quantidade máxima de alunos.

Editar aula da matriz curricular

Campos editáveis:

- Horário;
- Professor;
- Lista de cursos autorizados.

Regras:

- Não permitir edição que cause inconsistência (ex: horário inexistente).
- Alterações não devem remover alunos já matriculados.

Excluir aula da matriz curricular

Regras:

- A exclusão não é permitida se houver alunos matriculados.
 - A exclusão deve ser lógica

Aluno

Visualizar aulas matriculadas

Lista todas as aulas em que o aluno está matriculado

- Exibe:
 - Disciplina
 - Professor
 - Horário

Realizar matrícula

O aluno pode se matricular em uma aula disponível, desde que:

- A aula seja autorizada para o curso do aluno;
- Ainda existam vagas disponíveis;
- Não haja choque de horário com aulas já matriculadas.

Regras:

- O sistema deve garantir consistência em cenários concorrentes;
- Dois alunos não podem ocupar a mesma vaga simultaneamente.



Desafio Técnico

Desenvolvedor de Software

Página 4

Controle de Acesso

Coordenadores

- Não podem acessar dados de alunos que não estejam relacionados às suas matrizes.

Alunos

- Não podem visualizar matrizes ou matrículas de outros alunos;

Autenticação

- A autenticação e autorização deve ser feita utilizando Keycloak.

Pontos Importantes

- Controle de vagas deve ser transacional;
- Matrícula deve falhar caso qualquer regra seja violada;
- O choque de horário deve considerar:
 - Dia da semana;
 - Horário inicial e final.



Desafio Técnico

Desenvolvedor de Software

Página 5

Escopo do desafio

Este desafio tem como objetivo avaliar a capacidade do candidato em desenvolver e documentar uma aplicação moderna, seguindo boas práticas de backend, frontend e testes.

Entregáveis Obrigatórios

O candidato deverá entregar os seguintes artefatos:

Repositório Git

- Todo o código-fonte deve estar versionado em um repositório Git. O repositório deve conter:
 - Backend;
 - Frontend;
 - Documentação.
- Commits claros e objetivos serão considerados um diferencial.

Docker Compose

- Deve ser fornecido um arquivo **docker-compose.yml** contendo todas as dependências necessárias para execução da aplicação, incluindo:
 - Backend (Quarkus);
 - Frontend (Angular);
 - Keycloak;
 - Banco de dados (PostgreSQL).

Backend

- Stack obrigatória:
 - Linguagem: Kotlin (versão 2.0 ou superior) ou Java (versão 21 ou superior);
 - Framework: Quarkus (versão 3.20 ou superior);
 - Banco de dados: PostgreSQL;
 - Persistência: JPA/Hibernate ou Panache.
- Requisitos técnicos:
 - API REST seguindo boas práticas (HTTP status codes, verbos corretos);
 - Separação clara de camadas (ex: controller/resource, service);
 - Validação de regras de negócio no backend;



Desafio Técnico

Desenvolvedor de Software

Página 6

- Tratamento centralizado de erros;
- Controle de acesso por perfil (Aluno / Coordenador) no Keycloak.

Frontend

- Stack obrigatória:
 - Framework: Angular (versão 18 ou superior);
 - Estruturação do projeto com Angular router;
 - Proteção de rota;
 - Gerenciamento: Nx;
 - Programação reativa: RxJS;
 - UI Components: PrimeNG.
- Requisitos técnicos:
 - Backend e frontend devem ser projetos separados;
 - Comunicação via API REST;
 - Organização modular (libs / apps no Nx);
 - Uso adequado de RxJS (observables, streams, operadores);
 - Interface funcional (não é necessário foco em design avançado).

Documentação

- A documentação deve ser clara e suficiente para qualquer pessoa conseguir executar e testar o projeto.
- Deve conter:
 - Documentação da API;
 - Descrição de todos os endpoints;
 - Método HTTP;
 - Parâmetros de entrada;
 - Payloads de request e response;
 - Possíveis erros e códigos HTTP.
- Pode ser feita via:
 - Swagger / OpenAPI (preferencial)
 - Ou Markdown bem estruturado

Passo a passo para execução

- Pré-requisitos;
- Comando para subir a aplicação com Docker Compose;
- URLs de acesso:
 - Backend;



Desafio Técnico

Desenvolvedor de Software

Página 7

- Frontend;
 - Swagger.
- Testes Unitários
 - O backend deve conter testes unitários;
 - Não é necessário cobertura total, mas testes bem escolhidos;
 - Os testes unitários no frontend é opcional, mas se torna um diferencial;
 - Escolha as ferramentas de teste que preferir.



Desafio Técnico

Desenvolvedor de Software

Página 8

Critérios de Avaliação

Backend

- Clareza do domínio e modelagem.
- Implementação correta das regras de negócio.
- Organização do código.
- Qualidade e relevância dos testes unitários.

Frontend

- Organização do projeto (Nx).
- Uso correto de RxJS.
- Comunicação eficiente com o backend.
- Clareza e manutenibilidade do código.

Geral

- Capacidade de seguir o escopo proposto.
- Qualidade da documentação.
- Facilidade de execução do projeto.
- Aderência à stack solicitada.
- Histórico Git.



Desafio Técnico

Desenvolvedor de Software

Página 9

Considerações finais

Mais importante do que entregar todas as funcionalidades listadas é a qualidade da solução apresentada. Valorizamos um código funcional, bem estruturado, legível e coerente, que demonstra boas práticas de desenvolvimento, clareza de raciocínio e **cuidado com regras de negócio**.

É esperado que o desafio seja implementado de forma completa, porém entendemos que o tempo é limitado. Caso seja necessário fazer escolhas, priorize a elegância da solução, a organização do código e a correta implementação das regras principais, mesmo que alguma funcionalidade secundária fique incompleta ou simplificada. Uma solução bem pensada e consistente é sempre preferível a uma implementação extensa, porém frágil ou confusa.

Fique à vontade para documentar decisões técnicas, simplificações adotadas ou pontos que você gostaria de evoluir com mais tempo. Isso será visto de forma positiva e ajudará a entender seu processo de tomada de decisão como desenvolvedor.

Se surgir qualquer dúvida ao longo do desafio, seja sobre regras de negócio, escopo, stack ou expectativas, não hesite em entrar em contato. Estamos disponíveis para ajudar e esclarecer o que for necessário:

- caiocolares@unifor.br
- cynthiabravo@unifor.br
- gabriel.matos@unifor.br
- rafael.reis@unifor.br

Boa sorte, bom desenvolvimento e, principalmente, aproveite o desafio!